Data-Driven Logistics Optimisation

Strategic Planning Report: Late-Delivery Risk, Demand Forecasting and Route Optimisation using Python

---

Scenario: DataCo Global, an omni-channel retailer shipping clothing, sports and electronics supplies worldwide

Date: 28 October 2026

Primary dataset: DataCo Smart Supply Chain for Big Data Analysis (Mendeley Data, DOI 10.17632/8gx2fvg2k6.5)

Code: see the accompanying repository (src/analysis.py, src/routing.py, src/data_utils.py)

Executive Summary

Late deliveries, stock-outs and inefficient delivery routes are three of the most expensive and visible failures in logistics. They are linked: poor demand forecasts create the wrong inventory in the wrong place, which forces expensive expedited shipping, which in turn raises late-delivery rates and erodes customer satisfaction.

This report plans an analytics programme in Python that attacks all three problems with one connected workflow: (1) classification to predict which orders are likely to be delivered late, (2) regression-based demand forecasting feeding a safety-stock and reorder-point policy, (3) clustering to segment regions by service profile, and (4) mathematical optimisation (capacitated vehicle routing) to cut last-mile distance. Success is measured with five KPIs: on-time delivery rate, average delivery delay, forecast accuracy (MAPE), inventory service level / stock-out rate, and transport cost per order.

Note on results: the figures shown in Section 6 were produced on a synthetic dataset with the same schema as the public data, purely to prove that the code runs end to end. They are not findings about the real DataCo data and should be regenerated once the real CSV is loaded.

1. Background and Research

1.1 Why logistics analytics matters

Logistics typically accounts for a large share of a retailer's operating cost, and delivery reliability is a key driver of customer retention. Three decision areas are most amenable to data science: inventory (how much to hold and when to reorder), transportation (which route, mode and carrier), and service-level management (which orders are at risk). An academic study on delivery-time forecasting using this same dataset cites McKinsey estimates that AI in supply chain management can cut forecast errors by 20–50% and costs by 10–15% (Vestnik DSTU, secondary citation; to be verified against the primary source before any business case relies on it).

1.2 Public data sources reviewed

Dataset	Content	Use in this project
DataCo Smart Supply Chain (Mendeley Data / Kaggle)	Orders, scheduled vs real shipping days, shipping mode, region, customer segment, sales, profit, late-delivery flag; plus clickstream logs	Primary: KPIs, late-delivery classification, demand forecasting, clustering
M5 Forecasting (Walmart, Kaggle)	Hierarchical daily unit sales with prices and calendar events	Benchmark for forecasting methods
Olist Brazilian E-Commerce (Kaggle)	Orders, delivery timestamps, customer and seller geolocation	Optional validation of delivery-delay models
Solomon / Homberger VRP benchmarks	Standard vehicle-routing instances with known best solutions	Validate the routing solver

The DataCo dataset was selected as the primary source because it is free, widely used in published work (including delivery-time forecasting and sales and inventory forecasting studies), and contains both the target variable (Late_delivery_risk) and the commercial fields needed to translate model output into money. Its main limitations are that it is a single company's simulated/anonymised data, it has no explicit stock-level or vehicle data, and some fields (for example actual shipping days) are only known after delivery, which creates a data-leakage risk that the plan explicitly controls for.

1.3 Methodological literature

- **Inventory theory:** safety stock and reorder-point models under uncertain demand and lead time (Silver, Pyke and Thomas, Inventory and Production Management in Supply Chains).
- **Demand forecasting:** gradient-boosted trees with lag and calendar features performed strongly in the M5 competition (Makridakis et al., 2022), motivating their use over purely classical baselines.
- **Vehicle routing:** the CVRP is NP-hard; metaheuristics such as guided local search give high-quality solutions quickly (Toth and Vigo, The Vehicle Routing Problem; Google OR-Tools).

2. Project Definition

2.1 Logistics scenario

DataCo Global sells clothing, sports equipment and electronics to consumers, corporate and home-office customers across several markets. Orders ship under four service levels (Standard, Second Class, First Class, Same Day). Management has three recurring pain points:

- **Service reliability:** a material share of orders arrive later than the scheduled shipping window, and operations learn about it only after the fact.
- **Inventory imbalance:** demand is seasonal and varies by category and market, so stock is simultaneously over- and under-supplied.
- **Last-mile cost:** deliveries from regional hubs are planned manually, producing avoidable distance and vehicle use.

2.2 Objectives

- O1. Predict at order time which orders are at high risk of late delivery, so operations can intervene (re-route, upgrade mode, notify customer).
- O2. Forecast weekly demand per category with lower error than a naive baseline, and convert forecasts into reorder points and safety stock.
- O3. Identify groups of regions with similar service/profit profiles to target process improvements.
- O4. Quantify the distance and vehicle savings of optimised routes versus a manual baseline.
- O5. Deliver a reproducible Python pipeline and a KPI dashboard that decision-makers can refresh monthly.

2.3 Key Performance Indicators

KPI	Definition	Data fields	Target (proposed)
On-time delivery rate	% orders with real shipping days ≤ scheduled days	Days for shipping (real), Days for shipment (scheduled), Late_delivery_risk	+5 percentage points vs baseline
Average delivery delay	Mean of (real − scheduled) days, overall and by mode / region	Same as above	Reduce by 20%
Forecast accuracy (MAPE)	Mean absolute percentage error of weekly demand forecast on a held-out period	order date, Order Item Quantity	Beat seasonal-naive baseline; MAPE < 15%
Stock-out rate / service level	% of demand periods where demand exceeds stock; target cycle service level 95%	Derived from forecast + policy simulation	≥ 95% service level

KPI	Definition	Data fields	Target (proposed)
Transport cost per order	Route distance × cost per km + vehicle cost, divided by orders delivered	Hub/customer coordinates, vehicle parameters	Reduce by 15–30% in routing pilot

Targets are proposals to be calibrated once baselines are measured in Phase 2; they are not claims about current DataCo performance.

3. How Data Science Applies

Technique	Application	Python tools / evaluation
Classification	Predict Late_delivery_risk from order-time features (mode, region, segment, category, quantity, season)	scikit-learn HistGradientBoosting; ROC-AUC, precision/recall, cost-weighted threshold
Regression / time series	Weekly demand per category using lags, rolling means, calendar features	LinearRegression baseline, GradientBoosting; MAPE on rolling-origin split
Clustering	Group regions/markets by late rate, delay, margin to prioritise interventions	KMeans on standardised features; silhouette score
Optimisation	Capacitated vehicle routing; reorder-point policy under service-level constraint	Google OR-Tools; compare to naive route distance
Descriptive analytics	KPI tracking, delay root-cause analysis (e.g. by shipping mode)	pandas groupby, matplotlib

The techniques are chained rather than independent: the clustering identifies where problems concentrate, the classifier flags individual at-risk orders, the forecast sets inventory positions that reduce the need for expediting, and routing lowers the cost of the physical delivery. A key design principle is to use only features available at decision time (no post-delivery fields) in any predictive model.

4. Strategic Roadmap

The end-to-end flow has six phases. Durations assume one analyst working part-time; they are planning estimates.

Phase	Activities	Deliverables	Duration
1. Data collection	Download DataCo CSV (latin-1 encoding), document schema, store raw data read-only, set up Git repo and environment	Data dictionary, repo, requirements.txt	Week 1
2. Cleaning and baseline KPIs	Parse dates, remove duplicates and invalid sales, handle missing values, engineer delay_days, month, day-of-week; compute baseline KPIs	Clean dataset, KPI baseline table	Weeks 1–2
3. Exploratory analysis	Late rate by mode/region/segment, seasonality, correlation checks, outlier review; clustering of regions	EDA notebook, cluster profiles	Weeks 2–3
4. Predictive modelling	Late-risk classifier; demand forecast with time-based validation; leakage checks; hyper-parameter tuning	Trained models, evaluation report	Weeks 3–5
5. Optimisation	Reorder-point policy; CVRP routing pilot;	Policy table, route plans,	Weeks 5–6

Phase	Activities	Deliverables	Duration
	scenario comparison versus baseline	savings estimate	
6. Reporting and deployment	KPI dashboard, recommendations, monitoring plan (drift, monthly retrain)	Final report, dashboard, handover notes	Weeks 6–7

Risks and mitigations

- **Data leakage:** exclude post-delivery fields from model inputs; validate with time-based splits.
- **Class imbalance / threshold choice:** evaluate with precision-recall and choose the threshold using the cost of a missed late order versus the cost of an unnecessary intervention.
- **No real vehicle or stock data:** routing and inventory results rely on stated assumptions (capacity, lead time, cost per km); treat them as pilots and run sensitivity analysis.
- **Generalisability:** one company's data; validate on a second dataset (e.g. Olist) before generalising conclusions.

5. Code Illustration

The snippets below are condensed from the working scripts in the repository. Full versions, including imports and a synthetic-data fallback, are in src/.

5.1 Load and clean

```
df = pd.read_csv("data/DataCoSupplyChainDataset.csv", encoding="latin-1")
df["order_date"] = pd.to_datetime(df["order date (DateOrders)"])
df = df.drop_duplicates()
df = df[df["Sales"] > 0]
df["delay_days"] = df["Days for shipping (real)"] - df["Days for shipment (scheduled)"]
```

5.2 KPI computation

```
def compute_kpis(df):
    return pd.Series({
        "on_time_delivery_rate_%": 100 * (1 - df["Late_delivery_risk"].mean()),
        "avg_delay_days": df["delay_days"].mean(),
        "profit_margin_%": 100 * df["Order Profit Per Order"].sum() /
df["Sales"].sum(),
    })
df.groupby("Shipping Mode")["Late_delivery_risk"].mean().sort_values()
```

5.3 Late-delivery classification

```
# Leakage guard: use only information known AT ORDER TIME
cat = ["Shipping Mode", "Order Region", "Market", "Customer Segment", "Category Name"]
num = ["Days for shipment (scheduled)", "Order Item Quantity", "Sales", "month", "dow"]
Xtr, Xte, ytr, yte = train_test_split(df[cat+num], df["Late_delivery_risk"],
                                      test_size=.2, stratify=df["Late_delivery_risk"])
pre = ColumnTransformer([("c", OneHotEncoder(handle_unknown="ignore"), cat),
                         ("n", StandardScaler(), num)])
model = Pipeline([("pre", pre), ("clf", HistGradientBoostingClassifier())]).fit(Xtr,
ytr)
print(roc_auc_score(yte, model.predict_proba(Xte)[:, 1]))
```

5.4 Demand forecasting

```
weekly = df.set_index("order_date")["Order Item
Quantity"].resample("W").sum().to_frame("units")
for lag in (1, 2, 3, 4, 52): weekly[f"lag{lag}"] = weekly["units"].shift(lag)
weekly["roll4"] = weekly["units"].shift(1).rolling(4).mean()
train, test = weekly.dropna().iloc[:-8], weekly.dropna().iloc[-8:]
gbr = GradientBoostingRegressor().fit(train.drop(columns="units"), train["units"])
mape = mean_absolute_percentage_error(test["units"],
gbr.predict(test.drop(columns="units")))
```

5.5 Inventory policy

```
# Safety stock for ~95% cycle service level (z = 1.65), lead time L weeks
safety_stock  = 1.65 * std_weekly_demand * np.sqrt(L)
reorder_point = mean_weekly_demand * L + safety_stock
```

5.6 Clustering regions

```
profile = df.groupby("Order Region").agg(late=("Late_delivery_risk","mean"),
                                         delay=("delay_days","mean"),
                                         margin=("Order Profit Per Order","mean"))
profile["cluster"] = KMeans(n_clusters=3,
n_init=10).fit_predict(StandardScaler().fit_transform(profile))
```

5.7 Route optimisation (OR-Tools)

```
manager = pywrapcp.RoutingIndexManager(n_nodes, n_vehicles, depot=0)
routing = pywrapcp.RoutingModel(manager)
routing.SetArcCostEvaluatorOfAllVehicles(routing.RegisterTransitCallback(distance_cb))
routing.AddDimensionWithVehicleCapacity(routing.RegisterUnaryTransitCallback(demand_cb)
,
                                        0, [capacity]*n_vehicles, True, "Capacity")
params = pywrapcp.DefaultRoutingSearchParameters()
params.first_solution_strategy = FirstSolutionStrategy.PATH_CHEAPEST_ARC
params.local_search_metaheuristic = LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
solution = routing.SolveWithParameters(params)
```

6. Proof-of-Concept Run

To verify that the pipeline executes end to end, it was run on a 30,000-row synthetic dataset that mimics the DataCo schema. The left panel shows late rate by shipping mode; the right panel shows 30 customers served by five capacity-constrained vehicles. In the synthetic routing demo, the optimised plan covered a total distance of about 545 units versus about 1,528 for a naive single-truck sequence; this compares different setups (five trucks versus one) and only demonstrates that the solver works, not an expected saving.

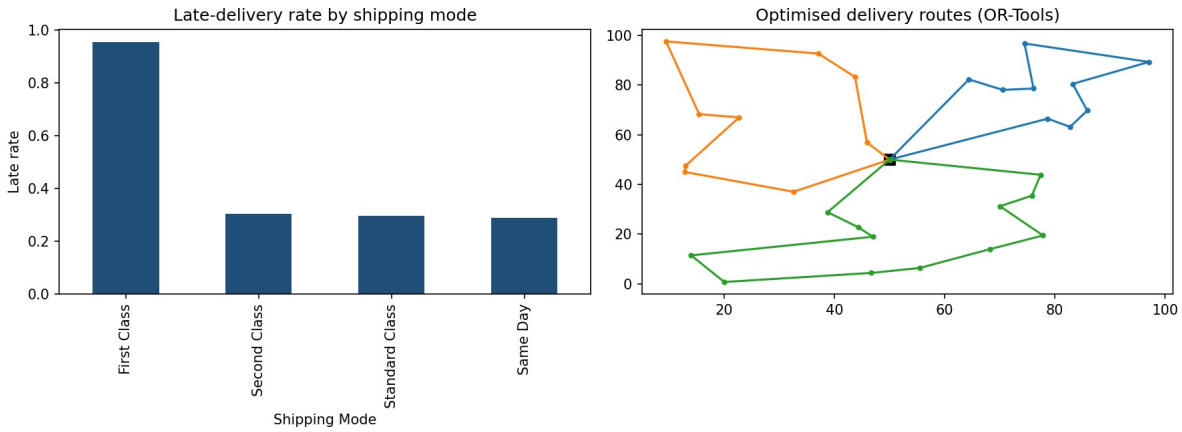


Figure 1. Output of src/analysis.py on synthetic data (illustrative only).

Other synthetic-run outputs: demand-forecast MAPE of about 8% (gradient boosting) and a late-delivery classifier ROC-AUC of about 0.68. These reflect the simulated data-generating process, so no business conclusion should be drawn from them.

7. Expected Outcomes and Impact on Decision-Making

- **Proactive service management:** a risk score at order time lets operations upgrade shipping, prioritise warehouse picking or warn customers before a delay happens, rather than reporting it afterwards.
- **Right-sized inventory:** forecast-driven reorder points and safety stock replace rule-of-thumb buffers, targeting a stated service level at lower working capital.
- **Targeted investment:** region clusters show where carrier renegotiation or hub changes will matter most.
- **Lower delivery cost:** optimised routes reduce distance, fuel and vehicles per delivery, and the same solver can be re-run daily.
- **Governance:** a version-controlled, reproducible pipeline with monitored KPIs lets managers trust, audit and refresh the analysis.

In conclusion, combining descriptive KPIs, predictive models and optimisation in one Python workflow turns logistics data from a reporting archive into a decision tool. The plan is feasible with free public data and open-source libraries, and the staged roadmap lets the team prove value early (baseline KPIs and the late-risk model) before investing in routing and inventory pilots.

References

1. Constante, F., Silva, F. and Pereira, A. (2019). DataCo Smart Supply Chain for Big Data Analysis. Mendeley Data, V5. DOI: 10.17632/8gx2fvg2k6.5. https://data.mendeley.com/datasets/8gx2fvg2k6/5
2. Forecasting Delivery Time of Goods in Supply Chains Using Machine Learning Methods. Vestnik of Don State Technical University. https://www.vestnik-donstu.ru/jour/article/view/2400
3. Forecasting Sales and Inventory in Supply Chain using Machine Learning (MSc research project using the DataCo dataset), National College of Ireland. https://norma.ncirl.ie/7251/2/riandwiputraconfigurationmanual.pdf
4. Makridakis, S., Spiliotis, E., Assimakopoulos, V. (2022). M5 accuracy competition: Results, findings, and conclusions. International Journal of Forecasting.
5. Silver, E.A., Pyke, D.F., Thomas, D.J. (2017). Inventory and Production Management in Supply Chains. CRC Press.
6. Toth, P. and Vigo, D. (2014). Vehicle Routing: Problems, Methods, and Applications. SIAM.

7. Google OR-Tools documentation, Vehicle Routing. https://developers.google.com/optimization/routing
8. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.